

Tema 4.3.3: Fundamentos de TypeScript y Sistema de Tipos Estático

Resumen

Este tema introduce **TypeScript**, un superconjunto de código abierto (*superset*) desarrollado por Microsoft que añade tipado estático opcional, inferencia de tipos y características avanzadas de Orientación a Objetos a JavaScript. Se analiza el proceso de transpilación mediante el compilador `tsc`, el archivo de configuración `tsconfig.json`, las anotaciones de tipo primitivas y compuestas, los tipos especiales (`any`, `unknown`, `never`), la creación de contratos de datos mediante `interface` y `type`, y el tipado estricto en funciones.

Concepto Principal (El 'Qué')

TypeScript es un superconjunto tipado de JavaScript que se transpila a JavaScript puro (ES5/ES6+) ejecutable en cualquier navegador o motor Node.js.

\$\$\text{TypeScript} = \text{JavaScript} + \text{Sistema de Tipos Estático} + \text{Herramientas de Compilación}\$\$

1. El Compilador `tsc` y `tsconfig.json`

A diferencia de JavaScript (que se interpreta en tiempo de ejecución), TypeScript analiza el código en tiempo de compilación. El archivo `tsconfig.json` rige las reglas del compilador:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "strict": true,
    "noImplicitAny": true,
    "skipLibCheck": true
  }
}
```

```
}
```

2. Anotaciones de Tipo Primitivas y Tipos Especiales

- **Primitivos:** `let placa: string, let peso: number, let activo: boolean.`
- **Arreglos y Tuplas:** `let lista: string[], let tupla: [number, string].`
- **any (El Escape Hatches - ¡Evitar!):** Desactiva la verificación de tipos para esa variable.
- **unknown (Tipo Seguro Desconocido):** Alternativa segura a `any`. Exige verificación de tipo (*Type Guard*) antes de realizar operaciones sobre la variable.
- **void y never:** `void` indica que una función no retorna valor, mientras que `never` representa funciones que nunca terminan (bucles infinitos o lanzamiento explícito de excepciones).

3. Contratos de Datos: `interface` VS `type`

Tanto `interface` como `type` permiten definir la estructura y forma (*shape*) de los objetos:

- **interface:** Orientada a objetos y extensible. Admite fusión de declaraciones (*declaration merging*) y la cláusula `extends`. Es el estándar recomendado para definir modelos DTO e interfaces de componentes en Angular.
- **type (Type Alias):** Más versátil para combinaciones de tipos de unión, primitivos o tuplas (`type EstadoEnvio = 'PENDIENTE' | 'EN_TRANSITO' | 'ENTREGADO'`).

```
// Definición de Interface para DTO de Dominio
export interface VehiculoDto {
  id: number;
  placa: string;
  capacidadKg: number;
  activo?: boolean; // Propiedad opcional
}
```

Analogía (La Intuición)

Imagine un plano arquitectónico blueprints frente a una maqueta de plastilina:

- **JavaScript Puro (Maqueta de Plastilina):** Es sumamente flexible. Usted puede moldear una plastilina azul como un auto y luego aplastarla para convertirla en una pelota en tiempo de ejecución. Sin embargo, no hay nada que le advierta si la pelota de plastilina colapsará estructuralmente hasta que le coloque peso encima (error en tiempo de ejecución).
- **TypeScript (Plano Arquitectónico Digital en CAD):** Antes de mover un solo gramo de material, el software CAD analiza las cargas estructurales de las vigas y calibra los tipos de acero exigidos en el plano (`string`, `number`, `VehiculoDto`). Si usted intenta colocar una viga de plástico donde se exige acero, el software marca un error rojo en el monitor del ingeniero en tiempo de diseño antes de ir a producción.

Ejemplo de Código e Implementación (El 'Cómo')

El siguiente ejemplo muestra la definición de interfaces, tipos de unión, enums y funciones fuertemente tipadas en TypeScript:

```
// 1. Enumeración para Estados de Negocio
export enum EstadoEnvio {
    PENDIENTE = 'PENDIENTE',
    EN_TRANSITO = 'EN_TRANSITO',
    ENTREGADO = 'ENTREGADO'
}

// 2. Interface de Entidad de Dominio
export interface Envio {
    id: number;
    trackingCode: string;
    pesoKg: number;
    estado: EstadoEnvio;
    descripcion?: string; // Campo opcional
}

// 3. Alias de Tipo para respuestas de API
export type RespuestaApi<T> = {
    exito: boolean;
    datos: T;
    timestamp: Date;
```

```

};

// 4. Función fuertemente tipada con valores por defecto
export const calcularFlete = (
  pesoKg: number,
  tarifaPorKg: number = 1500
): number => {
  if (pesoKg <= 0) {
    throw new Error("El peso debe ser mayor a cero.");
  }
  return pesoKg * tarifaPorKg;
};

// 5. Procesamiento tipado de objetos
export const procesarEnvio = (envio: Envio): string => {
  const costo = calcularFlete(envio.pesoKg);
  return `Envío #${envio.id} [${envio.estado}] - Costo Total: 🟩${costo}`;
};

```

Explicación Paso a Paso del Código:

1. **enum EstadoEnvio:** Define una lista restringida de constantes con nombre, garantizando que el estado del envío sólo adopte valores válidos.
2. **export interface Envio:** Contrato estricto que describe la forma del objeto Envio. Si falta una propiedad obligatoria (como `trackingCode`), TypeScript emitirá un error de compilación.
3. **descripcion?: string:** El signo `?` marca la propiedad como opcional (su tipo real es `string | undefined`).
4. **Tipado de Funciones:** Parámetros (`pesoKg: number`) y tipo de retorno explícito `: number`.

Trampas Comunes

Uso Abusivo del Tipo any (Any-script):

- *Error:* Anotar variables complejas con `: any` para evitar corregir errores de tipado, anulando por completo las ventajas del compilador de TypeScript.

- *Solución:* Utilizar `unknown`, interfaces explícitas o genéricos `<T>`. Activar `"noImplicitAny": true` en el `tsconfig.json`.

Confundir Tipos de Tiempo de Compilación con Tiempo de Ejecución:

- *Error:* Asumir que la interfaz `interface Vehiculo` continuará existiendo en el archivo JavaScript compilado.
- *Solución:* Entender que las interfaces y alias de tipo se eliminan por completo (*Type Erasure*) durante la transpilación a JavaScript.

Asignar Tipos Incorrectos a Respuestas de APIs HTTP:

- *Error:* Confiar ciegamente en que la respuesta de `fetch()` coincide con la interfaz declarada sin realizar aserciones ni validaciones runtime.
- *Solución:* Validar la respuesta del servidor o utilizar librerías de validación de esquemas runtime (como `Zod`).

Pregunta de Reflexión

En proyectos de software empresarial con equipos de desarrollo distribuidos:

¿De qué manera el uso de TypeScript e interfaces fuertemente tipadas reduce la necesidad de pruebas unitarias defensivas de tipo primitivo (ejemplo: validar que `typeof id === 'number'`) y cómo impacta la productividad en el refactoring de código masivo en comparación con JavaScript puro?